

리눅스 3.6 - Maple Tree

남궁명수

[Maple Tree]

Maple Tree는 리눅스 커널이 사용하는 B-Tree 계열의 범위 기반 자료구조다.

일반적인 트리는 하나의 키에 하나의 값을 연결하지만, Maple Tree는 하나의 주소 또는 연속된 주소 범위에 값을 연결하도록 설계되었다.

[일반적인 Map

키 10 → 값 A

키 20 → 값 B

키 30 → 값 C

[Maple Tree]

주소 범위 1000~1999 → 값 A

주소 범위 2000~2999 → 값 B

주소 범위 4000~4999 → 값 C

Maple Tree는 겹치지 않는 범위를 저장하는 데 최적화되어 있으며,

리눅스 커널에서는 프로세스의 VAM를 관리하는 것이 가장 중요한 사용 사례다.

[Maple Tree의 핵심 특징]

- B-Tree 계열의 다진 트리
- 하나의 노드에 여러 범위와 값을 저장
- 겹치지 않는 범위 관리에 최적화
- 특정 주소가 포함된 범위 탐색
- 이전·다음 범위 순회
- 비어 있는 주소 범위 검색
- RCU를 이용한 동시 읽기 지원
- CPU 캐시 효율을 고려한 구조

[1] Maple Tree와 프로세스 가상 주소 공간

[프로세스의 가상 주소 공간]

각 프로세스는 독립적인 가상 주소 공간을 가진다.

프로세스 A의 가상 주소 공간

낮은 주소



높은 주소

리눅스 커널은 프로세스의 전체 가상 주소 공간을 struct mm_struct로 표현한다.

```
struct mm_struct
```

하나의 mm_struct 안에는 해당 프로세스 주소 공간에 존재하는 여러 VMA를 관리하는 Maple Tree가 포함된다.

```
struct mm_struct
├─ 페이지 테이블 관련 정보
├─ 가상 주소 공간 통계
├─ 메모리 관리 Lock
└─ Maple Tree
    ├─ 코드 VMA
    ├─ 데이터 VMA
    ├─ 힙 VMA
    ├─ mmap VMA
    └─ 스택 VMA
```

정확히는 같은 가상 주소 공간을 공유하는 스레드들은 같은 mm_struct를 함께 참조한다.

커널 공식 문서도 각각의 mm 객체가 해당 주소 공간의 모든 VMA를 설명하는 Maple Tree를 포함한다고 설명한다.

[2] VMA란?

VMA(Virtual Memory Area)는 동일한 속성을 가진 역속된 가상 주소 범위를 표현하는 커널 객체다.

```
struct vm_area_struct {
    unsigned long vm_start; // 시작 가상 주소
    unsigned long vm_end;   // 끝 가상 주소

    pgprot_t vm_page_prot; // 페이지 보호 속성
    unsigned long vm_flags; // 읽기, 쓰기, 실행 등의 속성

    struct mm_struct *vm_mm; // 소속된 주소 공간
    struct file *vm_file;    // 파일 매핑이면 연결된 파일
};
```

예를 들어, 다음과 같은 VMA가 있다고 해보자.

VMA A
시작 주소: 0x400000
끝 주소: 0x410000
속성: 읽기 + 실행
용도: 프로그램 코드

VMA B
시작 주소: 0x600000
끝 주소: 0x610000
속성: 읽기 + 쓰기
용도: 프로그램 데이터

VMA의 주소 범위는 일반적으로 다음과 같이 해석된다.

$$vm_start \leq \text{가상 주소} < vm_end$$

즉, vm_start는 포함하고 vm_end는 포함하지 않는다.

VMA: 0x400000 ~ 0x410000

포함:
0x400000
0x400001
...
0x40FFFF

포함하지 않음:

0x410000

[3] Maple Tree가 저장하는 정보

프로세스의 VMA를 관리하는 Maple Tree는 가상 주소 범위와 해당 VMA 객체를 연결한다.

가상 주소 범위

↓

struct vm_area_struct 포인터

개념적으로 다음과 같이 저장된다.

0x400000 ~ 0x410000

→ 코드 VMA

0x600000 ~ 0x610000

→ 데이터 VMA

0x800000 ~ 0xA00000

→ 힙 VMA

0x7FFF0000 ~ 0x80000000

→ 스택 VMA

[Maple Tree]

[0x400000~0x40FFFF] → 코드 VMA

[0x600000~0x60FFFF] → 데이터 VMA

[0x800000~0x9FFFFFF] → 힙 VMA

[0x7FFF0000~0x7FFFFFFF] → 스택 VMA

Maple Tree에서 탐색 기준으로 사용하는 것은 가상 주소다.

저장된 값은 해당 주소 범위를 설명하는 struct vm_area_struct를 가리킨다.

Key 또는 Index

→ 가상 주소 및 가상 주소 범위

Value 또는 Entry

→ struct vm_area_struct 포인터

[4] Maple Tree가 저장하지 않는 것

[Maple Tree는 물리 페이지를 저장하지 않는다]

Maple Tree가 저장하는 것은 VMA 정보다.

다음 항목을 직접 저장하거나 관리하는 자료구조는 아니다.

- 실제 프로그램 데이터
- RAM의 물리 페이지
- 물리 프레임 번호
- PTE
- 페이지 테이블
- Swap 데이터

[Maple Tree]

- 이 가상 주소가 어떤 VMA에 포함되는가?
- 그 VMA는 읽기·쓰기·실행 중 무엇을 허용하는가?
- 익명 메모리인가, 파일 매핑인가?

페이지 테이블

- 이 가상 페이지가 어느 물리 프레임에 연결되는가?

RAM 물리 페이지

- 실제 데이터가 어디에 저장되어 있는가?

관계를 구분하면 다음과 같다.

프로세스의 가상 주소

↓

Maple Tree에서 VMA 검색

↓

이 주소가 정상적인 메모리 영역인지 확인

↓

페이지 테이블의 PTE 확인

↓

RAM의 물리 프레임 확인

Maple Tree는 가상 주소 영역의 의미와 속성을 찾기 위한 자료구조이고,
페이지 테이블은 가상 페이지와 물리 프레임의 실제 매핑을 찾기 위한 자료구조다.

[5] VMA와 페이지의 차이

VMA와 페이지는 서로 다른 개념이다.

VMA

→ 동일한 속성을 가진 가상 주소 범위

가상 페이지

→ 가상 주소 공간을 일정 크기로 나눈 단위

물리 프레임

→ RAM을 일정 크기로 나눈 단위

예를 들어, 크기가 16KiB인 VMA가 있고 페이지 크기가 4KiB라면 다음과 같이 여러 가상 페이지를 포함할 수 있다.

VMA: 0x1000 ~ 0x5000

크기: 16KiB

가상 페이지 1	0x1000 ~ 0x1FFF
가상 페이지 2	0x2000 ~ 0x2FFF
가상 페이지 3	0x3000 ~ 0x3FFF
가상 페이지 4	0x4000 ~ 0x4FFF

Maple Tree

→ 16KiB 전체 범위에 대응하는 VMA를 관리

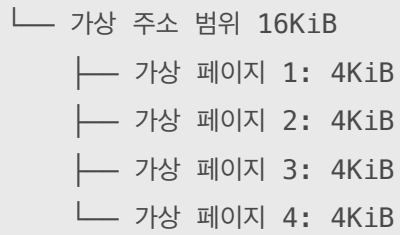
페이지 테이블

→ 각각의 4KiB 가상 페이지에 대한 매핑 관리

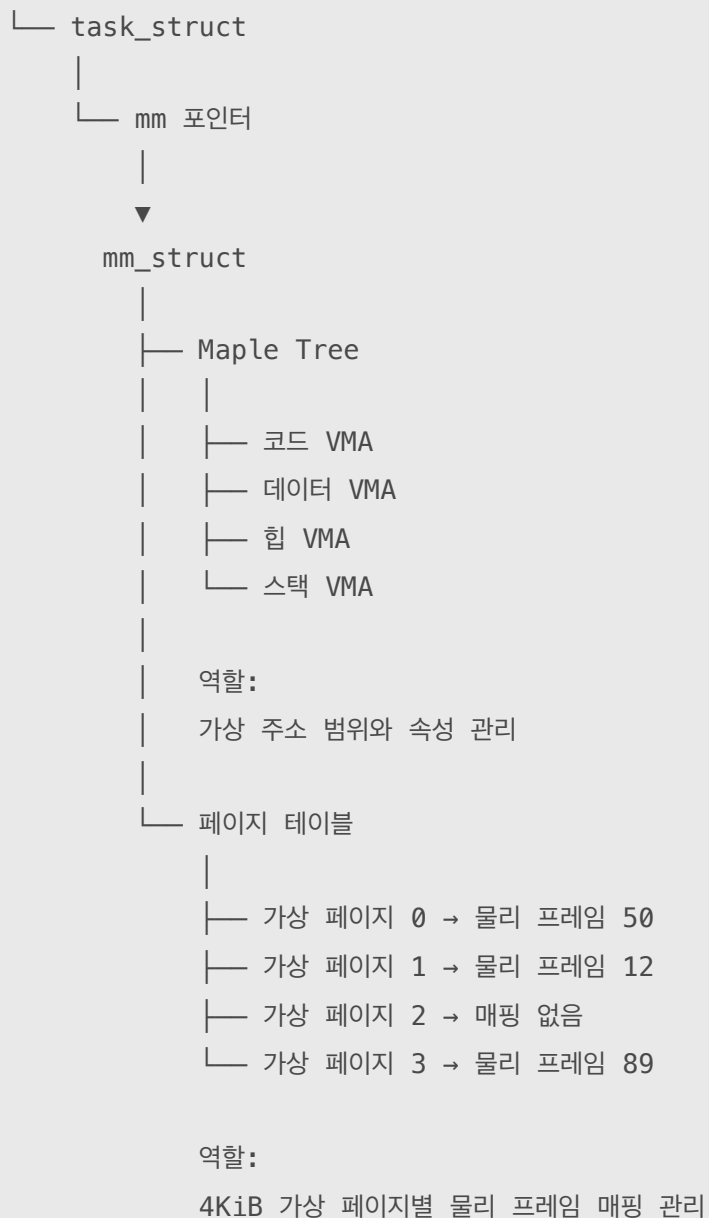
따라서 Maple Tree노드 하나가 물리 페이지 하나에 대응하는 것은 아니다.

[16KiB VMA와 4KiB 페이지의 관계]

VMA 하나



프로세스



[6] Maple Tree의 구조

[B-Tree 계열 구조]

Maple Tree는 Red-Black Tree처럼 노드 하나에 키 하나만 저장하는 이진 트리가 아니다.
하나의 노드에 여러 개의 경계값과 여러 개의 Entry 또는 자식 포인터를 저장하는 다진 트리다.

Red-Black Tree 노드

[키 하나]
/
왼쪽 \
오른쪽

Maple Tree 노드

[경계값 | 경계값 | 경계값 ...]
/ | | \
자식 자식 자식 자식

개념적으로 다음과 같은 모습이다.

[0x400000 | 0x800000 | 0xC00000]
/ | | \
낮은 범위 중간 범위 높은 범위 더 높은 범위

노드 하나가 여러 갈래로 나뉘기 때문에 Red-Black Tree보다 높은 분기 계수(Fan-out)를 가질 수 있다.

높은 Fan-out

↓
트리 높이 감소
↓
탐색 중 방문할 노드 수 감소

Maple Tree의 노드는 단순히 범위 시작 주소만 나열하는 구조가 아니다.

내부적으로 Pivot가 Slot을 사용해 범위와 자식 또는 Entry를 관리한다.

Pivot

→ 각 Slot이 담당하는 주소 범위의 경계

Slot

→ 다음 Maple 노드

또는

→ 저장된 Entry

개념적으로는 다음과 같다

```
Pivot: [1999 | 2999 | 4999]
Slot:  [ A   | B   | NULL | C ]
```

0~1999

→ Entry A

2000~2999

→ Entry B

3000~4999

→ 비어 있음

그 이후 일부 범위

→ Entry C

실제 내부 인코딩과 노드 형식은 더 복잡하지만, 학습할 때는 범위 경계 + 자식 또는 값을 한 노드에 여러 개 저장한다고 이해하면 된다.

[7] Maple Tree의 Entry

Entry는 Maple Tree가 해당 주소 범위에 연결한 값이다.

VMA 관리에서는 Entry가 struct vm_area_struct를 가리킨다.

가상 주소 범위

0x400000 ~ 0x410000

↓

Maple Tree Entry

↓

struct vm_area_struct

```
struct vm_area_struct
```

```
├─ vm_start
```

```
├─ vm_end
```

```
├─ vm_flags
```

```
├─ vm_page_prot
```

```
├─ vm_file
```

```
└─ 기타 VMA 관리 정보
```

Maple Tree 자체가 VMA 구조체의 모든 필드를 노드 안에 복사하는 것으로 이해하면 안된다. 개념적으로 Maple Tree에는 VMA를 가리키는 Entry가 저장되고, 실제 vm_area_struct 객체는 커널 메모리에 존재한다.

[8] 빈 주소 범위

프로세스의 가상 주소 공간 전체가 VMA로 채워져 있는 것은 아니다.

VMA 사이에는 사용하지 않는 빈 가상 주소 범위가 존재한다.

가상 주소 공간

[코드 VMA]
[빈 공간]
[데이터 VMA]
[힙 VMA]
[빈 공간]
[mmap VMA]
[빈 공간]
[스택 VMA]

Maple Tree는 Entry가 없는 범위를 NULL로 표현할 수 있다.

0x400000 ~ 0x410000 → 코드 VMA
0x410000 ~ 0x600000 → NULL
0x600000 ~ 0x610000 → 데이터 VMA

Maple Tree는 설정에 따라 특정 크기 이상의 빈 범위를 찾는 기능도 제공한다.

이는 새로운 mmap() 영역을 배치할 빈 가상 주소 범위를 찾는 작업에 유용하다.

mmap()이 1MiB 공간 요청

↓

Maple Tree에서 1MiB 이상인 빈 범위 검색

↓

적절한 가상 주소 범위 선택

↓

새로운 VMA 생성 및 등록

단, Maple Tree가 물리 메모리 1MiB를 찾는 것은 아니다.

찾는 대상

→ 빈 가상 주소 범위

찾지 않는 대상

→ RAM의 빈 물리 프레임

[9] Maple Tree의 탐색

[특정 가상 주소가 속한 VMA 찾기]

프로세스가 가상 주소 0x805000에 접근했다고 해보자.

현재 VMA가 다음과 같이 존재한다.

코드 VMA

0x400000 ~ 0x410000

데이터 VMA

0x600000 ~ 0x610000

힙 VMA

0x800000 ~ 0xA00000

Maple Tree는 0x805000이 포함된 범위를 검색한다.

검색 주소: 0x805000

↓

$0x800000 \leq 0x805000 < 0xA00000$

↓

힙 VMA 발견

반환된 VMA에서 접근 권한 등을 확인한다.

힙 VMA

└─ 읽기 가능

└─ 쓰기 가능

└─ 실행 불가

[10] Page Fault에서의 역할

프로세스가 아직 물리 페이지가 연결되지 않은 가상 주소에 처음 접근하면 Page Fault가 발생할 수 있다.

프로세스가 가상 주소에 접근
↓
CPU의 MMU가 페이지 테이블 확인
↓
유효한 PTE 매핑 없음
↓
Page Fault
↓
커널 Page Fault Handler 실행

커널은 우선 해당 가상 주소가 정상적인 VMA에 포함되어 있는지 확인해야 한다.

Fault 주소
↓
Maple Tree에서 VMA 검색
↓
VMA 존재 여부 확인

[VMA가 존재하는 경우]

Fault 주소가 VMA 안에 있음
↓
VMA의 접근 권한 확인
↓
정상적인 접근이라면 페이지 준비
↓
빈 물리 프레임 또는 파일 페이지 연결
↓
PTE 갱신
↓
중단된 명령 재실행

[VMA가 존재하지 않는 경우]

Fault 주소에 대응하는 VMA가 없음
↓
유효하지 않은 가상 주소 접근
↓
프로세스에 SIGSEGV 전달 가능

[VMA는 있지만 권한이 없는 경우]

예를 들어, 읽기 전용 VMA에 쓰기를 시도했다고 해보자.

VMA는 존재
↓
쓰기 권한 없음
↓
보호 위반
↓
프로세스에 SIGSEGV 전달 가능

중요한 점은 다음과 같다.

CPU의 MMU가 Maple Tree를 직접 탐색하는 것은 아니다.

CPU·MMU

→ 페이지 테이블과 TLB를 사용

리눅스 커널

→ Page Fault 처리 과정에서 Maple Tree를 사용해 VMA 검색

[11] VMA의 생성과 삭제

[mmap()과 Maple Tree]

프로세스가 mmap()을 호출하면 커널은 새로운 가상 주소 범위를 준비한다.

프로세스가 mmap() 호출
↓
커널이 빈 가상 주소 범위 검색
↓
새로운 VMA 생성
↓
VMA 시작·끝 주소와 속성 설정
↓
Maple Tree에 VMA 등록

예를 들어, 다음 요청이 있다고 하자.

```
mmap(..., 16 * 1024, PROT_READ | PROT_WRITE, ...);
```

개념적으로 다음 VMA가 생성될 수 있다.

새 VMA

주소 범위:

0x700000 ~ 0x704000

속성:

읽기 + 쓰기

Maple Tree에는 다음과 같이 등록된다.

0x700000 ~ 0x703FFF

→ 새로운 VMA

하지만 mmap() 직후 모든 가상 페이지에 물리 프레임이 연결되는 것은 아니다.

mmap()

→ 가상 주소 범위와 VMA 준비

처음 실제 접근

→ Page Fault

→ 필요한 물리 페이지 연결

[12] munmap()과 Maple Tree

프로세스가 munmap()으로 주소 범위를 해제하면 해당 범위를 관리하던 VMA가 삭제되거나 분할될 수 있다.

[VMA 전체를 해제하는 경우]

기존 VMA:

0x700000 ~ 0x704000

전체 munmap()

↓

Maple Tree에서 VMA 제거

[VMA 가운데 일부를 해제하는 경우]

기존 VMA:

0x700000 ~ 0x706000

해제 범위:

0x702000 ~ 0x704000

기존 VMA가 두 개로 나뉠 수 있다.

해제 전:

[0x700000 ————— 0x706000]

해제 후:

[0x700000 ~ 0x702000]

빈 공간

[0x704000 ~ 0x706000]

Maple Tree도 변경된 VMA 범위에 맞게 갱신된다.

[13] 인접 VMA의 병합

서로 인접한 VMA의 속성이 같고 병합 조건을 만족하면 커널이 하나의 VMA로 합칠 수 있다.

병합 전:

VMA A

0x1000 ~ 0x3000

읽기 + 쓰기

VMA B

0x3000 ~ 0x5000

읽기 + 쓰기

병합 후:

VMA C

0x1000 ~ 0x5000

읽기 + 쓰기

Maple Tree에는 병합된 하나의 주소 범위와 VMA가 저장된다.

[0x1000 ~ 0x4FFF] → VMA C

모든 인접 VMA가 무조건 병합되는 것은 아니다.

병합을 막을 수 있는 차이

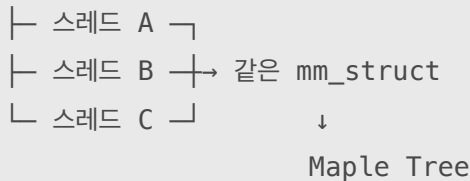
- 접근 권한
- 연결된 파일
- 파일 Offset
- 메모리 정책
- VMA별 특수 속성

[14] 동시성과 RCU

[여러 스레드의 VMA 탐색]

같은 프로세스의 스레드들은 가상 주소 공간을 공유하므로 같은 mm_struct와 Maple Tree를 참조한다.

프로세스



여러 스레드에서 동시에 Page Fault가 발생하면 VMA 탐색 역시 동시에 발생할 수 있다.

Maple Tree는 RCU-safe 모드를 지원하여 읽기와 쓰기가 동시에 존재하는 상황에서 확장성 있는 탐색을 지원할 수 있다.

기본 API에서는 읽기 작업에 RCU read lock을 사용하고,
갱신 작업에는 내부 Spinlock 또는 외부 Lock을 사용할 수 있다.

여러 Reader

→ 동시에 Maple Tree 탐색 가능

Writer

→ VMA 삽입·삭제·변경 시 동기화 필요

현대 커널의 VMA 관리에서는 mmap_lock, VMA 단위 Lock, RCU 등이 함께 사용된다.

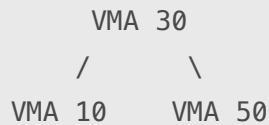
Maple Tree만으로 VMA의 모든 동시성 문제가 해결되는 것은 아니다.

[15] Red-Black Tree와의 차이

[기존 VMA 관리 방식]

과거의 리눅스 커널은 프로세스 VMA 탐색에 Red-Black Tree를 사용했다.

Red-Black Tree는 노드 하나에 VMA 하나를 연결하고 최대 두 개의 자식으로 분기한다.



또한 순차 탐색을 지원하기 위해 VMA들을 별도의 연결 구조로 관리하던 방식이 사용되었다.

개념적으로는 다음과 같다.

Red-Black Tree

→ 특정 주소의 VMA를 빠르게 탐색

연결 구조

→ 이전·다음 VMA 순회

현대 리눅스의 프로세스 주소 공간은 Maple Tree를 이용하여 VMA를 관리한다.

공식 문서도 각 mm_struct가 모든 VMA를 설명하는 Maple Tree를 포함한다고 명시한다.

[16] Maple Tree가 유리한 이유

[높은 Fan-out]

Red-Black Tree는 노드마다 최대 두 자식만 가진다.

Red-Black Tree

→ 최대 2갈래

Maple Tree는 노드 하나가 여러 개의 범위와 자식을 가질 수 있다.

Maple Tree

→ 여러 갈래

높은 Fan-out

↓

낮은 트리 높이

↓

적은 노드 접근

[CPU 캐시 효율]

Red-Black Tree는 노드마다 키 하나와 포인터를 저장하고 노드들이 메모리 곳곳에 흩어질 수 있다.

노드 A 접근
↓ 포인터
노드 B 접근
↓ 포인터
노드 C 접근

Maple Tree는 노드 하나에 여러 범위와 자식 정보를 모아 저장한다.

Maple 노드 하나를 캐시에 적재
↓
여러 경계값과 Slot을 함께 확인

공식 문서에 따르면 Maple Tree는 작은 메모리 사용량과 현대 CPU 캐시의 효율적인 사용을 고려해 설계되었으며, 노드 할당 크기는 대략 256바이트 수준이다.

[범위 관리]

VMA 자체가 하나의 가상 주소 범위를 표현한다.

VMA
→ vm_start부터 vm_end까지의 범위

따라서 개별 키뿐 아니라 범위를 직접 다루도록 설계된 Maple Tree가 VMA 관리에 잘 맞는다.

[이전, 다음 VMA 순회]

Maple Tree는 현재 위치를 추적하는 상태를 이용하여 이전 또는 다음 Entry로 이동할 수 있다.

현재 VMA
↓
다음 VMA
↓
다음 VMA

사용 방식은 연결 리스트처럼 보일 수 있지만, 실제로 모든 VMA가 별도의 연결 리스트 포인터로 연결되어 있다는 뜻은 아니다.

[빈 공간 탐색]

Maple Tree는 특정 크기 이상의 빈 주소 범위를 찾도록 설정할 수 있다.

필요한 크기: 1MiB
↓
Maple Tree에서 사용되지 않는 범위 검색
↓
새로운 mmap 주소 결정

[17] Red-Black Tree와 Maple Tree 비교

구분	Red-Black Tree	Maple Tree
계열	균형 이진 탐색 트리	B-Tree 계열 다진 트리
노드당 자식	최대 2개	여러 개
주요 탐색 대상	개별 키	개별 인덱스와 연속 범위
균형 유지	색상 변경과 회전	B-Tree 계열 노드 재구성
트리 높이	상대적으로 높음	높은 Fan-out으로 낮음
캐시 효율	포인터 추적이 많을 수 있음	한 노드에 여러 정보 저장
범위 관리	별도 구현 필요	범위 저장에 최적화
빈 범위 검색	별도 정보·구현 필요	Allocation Tree 모드에서 지원
이전·다음 순회	연결 구조가 유리	반복자 상태를 통해 지원
동시 읽기	별도 동기화 필요	RCU-safe 모드 지원
VMA 관리	과거의 주요 구조	현대 커널의 주요 구조

[18] B+Tree와 Maple Tree의 차이

[둘 다 B-Tree 계열이지만 목적이 다르다]

Maple Tree와 B+Tree는 모두 하나의 노드에 여러 키와 자식을 저장하는 B-Tree 계열 구조지만 사용 목적이 다르다.

구분	B+Tree	Maple Tree
대표 사용처	데이터베이스 인덱스	리눅스 커널 내부
주요 저장 위치	SSD/HDD, 필요 시 RAM에 적재	커널 RAM
탐색 기준	데이터베이스 인덱스 키	정수 인덱스·가상 주소 범위
저장 값	레코드 또는 레코드 위치 정보	커널 객체 포인터
리프 구조	리프 노드끼리 연결	반복자 상태로 이전·다음 Entry 탐색
주요 최적화	저장장치 페이지 I/O	CPU 캐시와 범위 탐색
VMA 관리	사용하지 않음	대표적인 사용 사례

B+Tree를 그대로 가져와 VMA를 저장하는 것이 아니라, 비중첩 범위와 커널 동시성 환경에 맞게 별도로 설계된 자료구조다.

[19] Maple Tree와 Page Fault 전체 흐름

[가상 주소 접근 과정]

프로세스가 힙의 가상 주소에 처음 쓰기를 수행한다고 해보자.

```
int *number = malloc(sizeof(int));  
*number = 10;
```

전체 과정은 다음과 같다.

1. malloc()
 - 힙 가상 주소 범위 확보
 - 해당 범위를 설명하는 VMA가 존재
2. 프로그램이 가상 주소에 처음 쓰기
3. CPU의 MMU가 TLB와 페이지 테이블 확인
4. 유효한 PTE 매핑이 없으면 Page Fault 발생
5. 커널 Page Fault Handler 실행
6. 커널이 Maple Tree에서 Fault 주소에 해당하는 VMA 검색
7. VMA가 존재하고 쓰기 권한이 있는지 확인
8. 익명 메모리에 사용할 물리 프레임 준비
9. PTE에 가상 페이지 → 물리 프레임 매핑 기록
10. CPU가 중단됐던 쓰기 명령 재실행
11. 실제 RAM 물리 프레임에 값 10 저장

구성 요소별 역할은 다음과 같다.

구성 요소	역할
mm_struct	프로세스 전체 가상 주소 공간 표현
Maple Tree	주소에 해당하는 VMA 탐색
vm_area_struct	가상 주소 범위와 권한·매핑 속성 표현
페이지 테이블	가상 페이지와 물리 프레임 연결
PTE	개별 가상 페이지의 매핑 및 권한 정보
물리 프레임	실제 데이터 저장
CPU·MMU	주소 변환 시도와 Page Fault 발생
커널	VMA 확인, 페이지 준비, PTE 갱신

[20] 주의할 점

[Maple Tree가 Red-Black Tree를 모두 대체한 것은 아니다]

Maple Tree가 리눅스 커널의 모든 Red-Black Tree를 대체한 것은 아니다.

대체된 주요 대상

→ 프로세스 주소 공간 안의 VMA를 주소순으로 관리하는 구조

여전히 사용될 수 있는 구조

→ 파일 매핑의 Reverse Mapping

→ 익명 메모리 Reverse Mapping

→ 타이머, 스케줄링, 네트워크 등의 다른 커널 자료구조

실제로 최신 VMA 구조에도 파일 기반 매핑 등을 위한 Reverse Mapping 경로에서는 Red-Black Interval Tree 관련 필드가 존재할 수 있다.

이는 Maple Tree의 정방향 VMA 탐색과 목적이 다르다.

Maple Tree

프로세스 가상 주소 → VMA 탐색

Reverse Mapping 구조

파일·익명 Folio → 이 메모리를 매핑한 VMA 탐색

[21] 시간 복잡도

Maple Tree는 균형 잡힌 B-Tree 계열이므로 주요 탐색, 삽입, 삭제 연산은 트리 높이에 비례한다.

노드당 분기 수를 M , 저장된 Entry 수를 N 이라고 하면 개념적인 복잡도는 다음과 같다.

연산	시간 복잡도
특정 주소 Entry 탐색	$O(\log_m N)$
범위 삽입	$O(\log_m N)$
범위 삭제	$O(\log_m N)$
다음·이전 Entry 이동	상태를 활용해 효율적으로 수행
전체 Entry 순회	$O(N)$

시간 복잡도 표기상으로는 일반적으로 다음처럼 표현할 수 있다

탐색: $O(\log N)$

삽입: $O(\log N)$

삭제: $O(\log N)$

실제 성능에서는 낮은 트리 높이뿐 아니라 CPU 캐시 효율, 노드 밀도, 동시 읽기 방식도 중요하다.

자료구조:

- 리눅스 커널에서 사용하는 B-Tree 계열의 범위 기반 자료구조

주체:

- 리눅스 커널

주요 대상:

- 겹치지 않는 정수 범위
- 프로세스의 가상 주소 범위
- 해당 주소 범위를 설명하는 VMA

저장 위치:

- Maple Tree 노드와 VMA 객체는 커널 RAM에 저장

mm_struct와의 관계:

- 하나의 프로세스 가상 주소 공간을 mm_struct가 표현
- mm_struct 안의 Maple Tree가 해당 주소 공간의 VMA를 관리
- 같은 주소 공간을 공유하는 스레드는 같은 mm_struct를 사용

Key:

- 가상 주소 또는 가상 주소 범위

Entry:

- 해당 범위를 설명하는 struct vm_area_struct 포인터

저장하지 않는 것:

- 실제 프로그램 데이터
- 물리 페이지
- 물리 프레임
- PTE
- 페이지 테이블

CPU와의 관계:

- CPU·MMU가 Maple Tree를 직접 탐색하지 않음
- CPU·MMU는 TLB와 페이지 테이블을 사용
- Page Fault가 발생하면 커널이 Maple Tree를 사용해 VMA를 탐색

VMA 탐색 목적:

- 해당 가상 주소가 유효한 영역인지 확인
- 읽기·쓰기·실행 권한 확인
- 익명 메모리인지 파일 기반 메모리인지 확인
- Page Fault 처리 방향 결정

Red-Black Tree보다 유리한 점:

- 높은 Fan-out
- 낮은 트리 높이
- CPU 캐시 효율
- 연속된 주소 범위 관리
- 빈 주소 범위 탐색
- 이전·다음 Entry 순회
- RCU를 활용한 동시 읽기

주요 연산:

- 특정 주소의 VMA 검색
- 새로운 VMA 삽입
- VMA 삭제
- VMA 분할과 병합
- 이전·다음 VMA 순회
- 빈 가상 주소 범위 검색